Ejercicio práctico: Node.js + Express

Mini API de la Cafetería del CRUI

Prácticas Profesionalizantes I · Tiempo: 30 minutos · En parejas o con tu grupo del proyecto

---

Contexto

La cafetería del CRUI quiere mostrar su menú en una página web. Antes de hacer la parte visual necesita un **backend**: un servidor que entregue los productos en formato JSON y que permita agregar productos nuevos.

Tu tarea es construir ese servidor con **Node.js y Express**. Es exactamente la misma pieza que van a necesitar en el backend del proyecto grupal.

Antes de empezar, verificá: abrí una terminal y escribí `node -v`. Si aparece un número de versión, estás listo. Si no, trabajá junto a un compañero que lo tenga instalado.

Parte 1 — Preparar el proyecto (5 min)

1. Creá una carpeta llamada `cafeteria-api` y abrila en VS Code.
2. Abrí la terminal de VS Code (menú "Terminal → New Terminal") y ejecutá:

```
npm init -y
npm install express
```

3. Creá un archivo `.gitignore` con esta única línea:

```
node_modules/
```

4. Creá el archivo `index.js` (ahí va todo tu código).
5. **Recomendado:** en `package.json`, dentro de `"scripts"`, agregá la línea `"dev": "node --watch index.js"`. Así, con `npm run dev` el servidor se reinicia solo cada vez que guardás.

Parte 2 — Encender el servidor (5 min)

En `index.js`:

- Creá una aplicación de Express que escuche en el **puerto 3000**.
- Definí la ruta **GET /** que responda con el texto: `API de la Cafetería del CRUI funcionando`
- Encendé el servidor (`node index.js` o `npm run dev`) y abrí `http://localhost:3000` en el navegador.

> *Si ves el mensaje en el navegador, ¡tu servidor funciona!*

Parte 3 — Listar los productos (7 min)

Copiá este array en tu `index.js`. Es la "base de datos" de la cafetería (vive en memoria):

```
const productos = [
  { id: 1, nombre: 'Café con leche', precio: 1800, categoria: 'bebida' },
  { id: 2, nombre: 'Medialuna', precio: 900, categoria: 'comida' },
  { id: 3, nombre: 'Tostado de jamón y queso', precio: 3500, categoria: 'comida' },
  { id: 4, nombre: 'Agua mineral', precio: 1200, categoria: 'bebida' }
];
```

- Creá la ruta **GET /api/productos** que devuelva todos los productos **en formato JSON**.

- Probala en el navegador: `http://localhost:3000/api/productos`

Parte 4 — Buscar un producto por id (6 min)

- Creá la ruta **GET /api/productos/:id** que devuelva solo el producto con ese id.
- Si el producto **no existe**, respondé con código **404** y este JSON: `{ "error": "Producto no encontrado" }`

> ***Pista:*** *`req.params.id` llega como **texto** ("2"), pero los ids del array son **números** (2). ¿Cómo lo convertís?*

Parte 5 — Agregar un producto (7 min)

- Creá la ruta **POST /api/productos** que reciba en el "body" un producto como este:

```
{ "nombre": "Alfajor", "precio": 1100, "categoria": "comida" }
```

- **Validá** los datos: `nombre` es obligatorio y `precio` tiene que ser un **número mayor a 0**. Si algo no cumple, respondé con código **400** y un JSON con el error.
- Si los datos son válidos: asignale un **id nuevo** (uno más que el último), agregalo al array y respondé con código **201** y el producto creado.

> ***Pista:*** *para poder leer `req.body` necesitás un "middleware" que vimos en la presentación. Va antes de las rutas.*

¿Cómo pruebo el POST?

El navegador solo hace pedidos GET desde la barra de direcciones. Para el POST tenés dos opciones:

Opción A — Thunder Client (extensión de VS Code): "New Request" → elegí **POST** → URL `http://localhost:3000/api/productos` → pestaña **Body** → **JSON** → pegá el producto → "Send".

Opción B — Consola del navegador: abrí `http://localhost:3000`, presioná **F12**, pestaña **Console**, y pegá:

```
fetch('/api/productos', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nombre: 'Alfajor', precio: 1100, categoria: 'comida' })
})
  .then(res => res.json())
  .then(data => console.log(data));
```

Lista de pruebas: ¿mi API funciona?

Hacé estas pruebas **en orden** y marcá cada una:

#	Pedido	Resultado esperado
1	GET /	Texto de bienvenida
2	GET /api/productos	Lista con los 4 productos
3	GET /api/productos/2	Solo la Medialuna
4	GET /api/productos/99	Código 404 y el mensaje de error
5	POST /api/productos con el Alfajor	Código 201 y el Alfajor con id 5
6	POST con precio "1500" (entre comillas, texto)	Código 400 y un mensaje de error
7	GET /api/productos	Ahora aparecen 5 productos

Desafíos extra (si terminás antes)

- **Bonus 1 — Filtro:** que `GET /api/productos?categoria=bebida` devuelva solo las bebidas. (Pista: `req.query`)
- **Bonus 2 — Borrar:** creá `DELETE /api/productos/:id` que elimine el producto (404 si no existe).
- **Bonus 3 — Resumen:** creá `GET /api/resumen` que devuelva `{ "cantidad": 4, "precioPromedio": 1850 }` calculado a partir del array.

Si algo falla

Mensaje o síntoma	Causa probable
Cannot find module 'express'	No ejecutaste `npm install express` en esta carpeta
EADDRINUSE … :3000	Ya tenés un servidor encendido: cortalo con Ctrl+C
req.body es undefined	Falta `app.use(express.json())` o el header Content-Type
Cambio el código y no pasa nada	Reiniciá el servidor o usá `npm run dev`
Cannot GET /api/…	La ruta o el método no coinciden con lo que definiste
La búsqueda por id siempre da 404	Estás comparando el texto "2" con el número 2

Al terminar

Mostrale al docente las pruebas 1 a 7 funcionando. Si querés, subí el proyecto a un repositorio de GitHub (¡sin `node_modules`!). Lo que hiciste hoy es la base del backend de tu proyecto grupal.